

AI Portfolio Advisor: Assignment 2 Report

Pau Gratacos Fuste | ESADE - Prototyping with LLMs | March 2026

1. Main Questions Answered in This Second Prototype

Assignment 1 feedback noted: "limited detail on model evaluation, feature engineering, and empirical performance." Assignment 2 was driven by two guiding questions that directly address that feedback while adding a second, non-straightforward LLM feature as required.

Q1: How can multiple non-trivial LLM patterns coexist in one application?

Assignment 1 used a single LLM call pattern. Assignment 2 implements four distinct, non-straightforward integration patterns that go well beyond simple prompt-and-display:

- **Agentic tool-use chatbot (Sonnet):** A multi-turn chatbot where Claude autonomously decides which of 6 tools to call (portfolio summary, asset price, risk calculation, what-if analysis, market news, macro prediction). Python executes each tool server-side and feeds results back across up to 8 LLM turns. This is a stateful, multi-call loop with tool orchestration -- the core new LLM feature for Assignment 2.
- **Orchestrated multi-step decision chain (Sonnet):** AI Autopilot: (1) Claude proposes 3 trades as validated JSON, (2) Python simulates each trade computing new volatility/Sharpe/drawdown via `what_if_analysis`, (3) Claude receives all simulation results plus 5 cross-component signals and synthesises a ranked recommendation. This chain combines LLM reasoning with Python computation across multiple calls -- not a simple prompt.
- **Structured JSON generation (Sonnet):** The Stress Test view generates per-holding drawdown parameters as validated JSON. Python parses, validates schema, and runs a quantitative simulation engine -- the LLM output feeds computation, not display.
- **Context-enriched single-call (Haiku):** Portfolio advice and news summaries use dynamically constructed prompts enriched with 5+ quantitative signals (macro probability, NLP sentiment, efficient frontier weights, stress test losses, backtest metrics) -- far beyond a static template.

Q2: How can ML models feed LLM prompts with quantitative rigour?

The key architectural addition is a cross-component signal pipeline using `st.session_state` as a shared registry. Five views compute and store signals: macro correction probability (ML classifier), portfolio-weighted NLP sentiment (TF-IDF + LR), efficient frontier optimal weights (scipy SLSQP), stress test worst-case losses (LLM + simulation), and backtest performance (cumulative returns vs SPY/60-40). Every LLM prompt is dynamically enriched with all available signals, meaning advice quality improves as the user explores more tabs. This is non-trivial Python post-processing: aggregating ML model outputs, optimization solvers, and simulation engines into structured prompt context that the LLM can reason over.

2. Main Difficulties Found and How They Were Resolved

- **Train-serve feature skew:** Macro feature engineering was duplicated in 3 files, risking inconsistencies between training and inference. Solved by creating `ml_pipeline/features.py` as a single source of truth with versioned feature lists (v1: 12 features for deployed model, v2: 16 features with interaction/lag terms). All consumers import from this shared module, eliminating skew.
- **Class imbalance (~9% positive rate):** Market corrections are rare events. Gradient Boosting achieved 85% accuracy by predicting 'stable' always (AUC=0.51, useless). Addressed via `class_weight='balanced'` (LR/RF) and `scale_pos_weight` (XGBoost). Random Forest won the 4-model GridSearchCV comparison with AUC=0.719. Added contextual captions under every ML chart explaining why low AP and poor calibration are expected for rare-event prediction -- not signs of a broken model.
- **Temporal data leakage in sentiment model:** The original model used random split, allowing 'future' tweets in training. Switched to sequential (time-ordered) split. Test accuracy dropped from ~80% to ~61%, which is more honest -- the model now shows genuine out-of-sample generalization. GridSearchCV over C, max_features, and ngram_range was added to maximize performance under this stricter regime.
- **Multi-turn tool-use reliability:** The agentic chatbot initially looped infinitely when Claude requested tools with malformed inputs. Solved by capping iterations at 8, adding input validation in each tool executor, and returning structured JSON errors that Claude can recover from on the next turn.
- **Cross-component signal availability:** LLM prompts need signals from tabs the user may not have visited. Solved with graceful degradation: each signal block is wrapped in try/except checking `session_state`. Prompts adapt their richness to whatever data is available rather than failing or showing empty fields.

- **yfinance API instability:** The DX-Y.NYB (Dollar Index) ticker intermittently fails. Implemented a fallback chain (DX-Y.NYB -> DX=F) in both training and inference, plus individual ticker downloads to avoid MultiIndex parsing issues with batch downloads.

3. How AI Was Leveraged

AI was used as a collaborative engineering partner with a clear division of labour. I authored the core ML pipelines, Streamlit views, and application architecture from scratch. AI assisted in three specific capacities:

Architecture & Codebase Audit

Claude Code (Opus) systematically audited the codebase and identified improvement opportunities: code duplication across feature engineering, disconnected ML/LLM components, missing quantitative metrics (CVaR, dynamic risk-free rate), and shallow LLM prompt context. This produced a prioritized action plan that guided the implementation of the cross-component signal pipeline.

Implementation Assistance

Claude Code implemented improvements under my direction: creating the centralized feature module, expanding ML training pipelines with XGBoost and broader hyperparameter grids, building the agentic chatbot tool-use loop, enriching LLM prompts with 5 quantitative signals, adding CVaR/Sharpe/Max Drawdown calculations, and the AI Autopilot multi-step chain. Each change was reviewed and tested iteratively -- several bugs were caught during this process (v2 features passed to a v1 model, float values treated as dicts in eval artifacts, model loading order issues in Streamlit).

Documentation & ML Transparency

AI drafted contextual interpretation captions for every ML evaluation chart, explaining class imbalance effects, calibration limitations of tree models, and cross-validation variance from market regime shifts. The ML Transparency tab (10 interactive visualizations with contextual explanations) was built collaboratively. The README was rewritten to accurately reflect the current architecture, including the complete cross-component signal pipeline and 4 LLM pattern descriptions.

Repository: https://github.com/paugratacosfuste/portfolio_manager

Deployed App: <https://portfolio-manager-pau.streamlit.app/>